CHAPTER 5: IMPLEMENTATION, TESTING AND RESULTS

HavenSec Arena

AI-Powered Multi-Agent Red Team & Blue Team Cybersecurity Simulation Platform

> **Implementation-status note:** The available project material defines the proposed architecture and validation framework, but does not provide verified execution logs, final screenshots, or measured benchmark values. Accordingly, this chapter presents a rigorous implementation and testing structure without inventing numerical results. Actual observations can be inserted after controlled execution.

5.1 Introduction

This chapter presents the implementation approach, prototype workflow, testing strategy and result-evaluation framework for HavenSec Arena. The previous chapters established the problem, reviewed related systems, selected the AI-assisted multi-agent approach and defined the proposed architecture, modules, technology stack and development methodology.

The implementation is organized around a controlled simulation lifecycle: scenario initialization, sandbox preparation, Red Team simulation, event and telemetry generation, Blue Team detection and response, Referee evaluation, incident-timeline construction and report generation. The design maintains a strict boundary between simulated activity and real-world infrastructure.

The objective of implementation is not to perform unrestricted offensive activity. Instead, the prototype should demonstrate that a repeatable cybersecurity exercise can be coordinated, observed and evaluated using synthetic data and isolated resources.

5.2 Implementation Environment

Component	Technology / Specification	Purpose
Backend	Python 3.x	Simulation logic, orchestration and data processing
API Layer	FastAPI	Communication between dashboard and backend services
Frontend	React	Interactive dashboard and scenario interface
UI Styling	Tailwind CSS	Consistent and responsive interface styling
Sandbox	Docker	Isolation and reproducibility of simulation components
AI Layer	IBM Bob / selected AI service	Proposed reasoning and coordination assistance
Data	Synthetic / dummy data	Safe simulation inputs and telemetry
Hardware	64-bit computer; 16 GB RAM recommended	Development, testing and execution

Table 5.1: Implementation Environment

5.3 Implementation of Major Modules

5.3.1 Scenario Management Module

The Scenario Management Module is the entry point for an exercise. It accepts the scenario type, difficulty level, objectives and sandbox configuration, then creates the initial simulation state. The resulting context is passed to the Orchestrator so that all participating components operate against the same scenario definition.

- Select a predefined cybersecurity scenario.
- Set difficulty and simulation objectives.
- Prepare synthetic resources and initial state.
- Assign the scenario context to the participating agents.

5.3.2 Orchestrator Module

The Orchestrator controls the exercise lifecycle. It initializes the participating agents, maintains scenario state, sequences permitted simulation actions and provides controlled communication between the Scenario Manager, Red Team, Blue Team, Referee and reporting components.

5.3.3 Red Team Simulation Module

The Red Team module represents attacker behaviour through controlled simulation. Its actions are restricted to the selected scenario and sandbox. Reconnaissance, path planning and simulated execution may be represented as events rather than unrestricted interaction with real systems.

5.3.4 Blue Team Defense Module

The Blue Team consumes telemetry generated by the sandbox. It analyzes suspicious events, records detection time and performs or recommends predefined defensive actions. Containment and recovery information are retained for subsequent evaluation.

5.3.5 Referee and Scoring Module

The Referee evaluates the exercise independently of the simulated attacker and defender. It uses timestamped events and recorded actions to derive detection time, containment, simulated damage and recovery indicators. These values are combined using a predefined scoring model.

5.3.6 Timeline and Reporting Module

The reporting layer orders timestamped events and agent actions into an incident timeline. It combines the timeline with evaluation metrics to produce findings, observations and improvement recommendations.

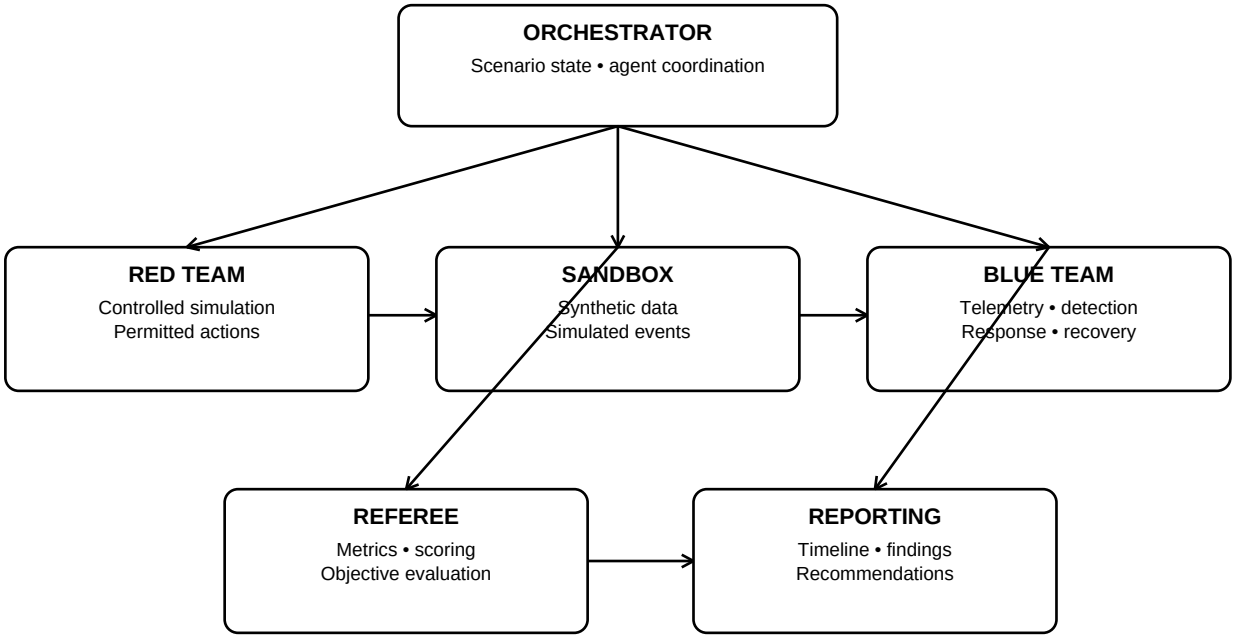


Figure 5.1: Implemented-module interaction model

5.4 End-to-End Implementation Workflow

The implementation follows the lifecycle defined in Chapter 4. Each step produces a traceable output that can be consumed by the next stage. The workflow is deliberately constrained so that simulation actions remain inside the configured sandbox.

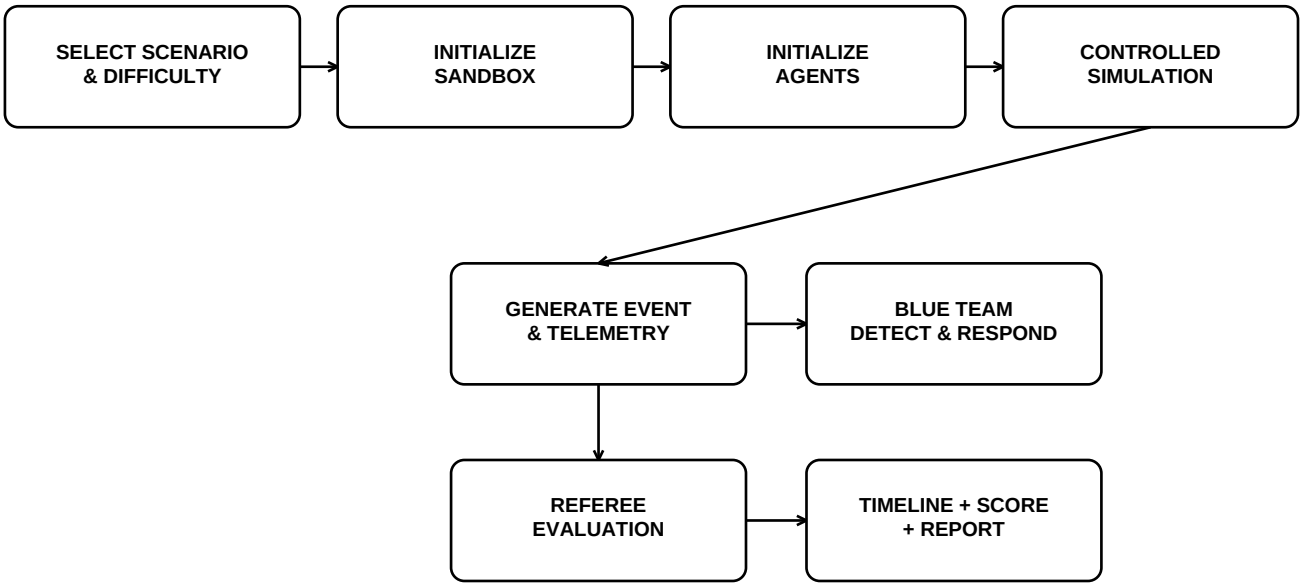


Figure 5.2: End-to-End Implementation Workflow

Step	Implementation Activity	Expected Output
1	Select scenario and difficulty	Scenario configuration
2	Initialize sandbox with synthetic data	Isolated simulation state
3	Initialize Orchestrator and agents	Active exercise components
4	Red Team selects permitted action	Controlled simulated action
5	Execute action inside sandbox	Simulation event
6	Record event, telemetry and timestamp	Event record
7	Provide telemetry to Blue Team	Detection input
8	Blue Team analyzes and responds	Detection / response record
9	Referee evaluates exercise	Evaluation metrics
10	Calculate resilience score	Composite score
11	Generate timeline and report	Incident report
12	Display observations	Improvement feedback

Table 5.2: End-to-End Implementation Workflow

5.5 Data and Event Handling

The logical data model defined in Chapter 4 provides a consistent structure for simulation state, agent actions and evaluation results. Event traceability is important because the Referee requires objective evidence for timing, containment and recovery calculations.

Entity	Key Information	Use in Testing
Scenario	scenario_id, name, difficulty, objective, status	Identifies exercise configuration
Agent	agent_id, role, type, status	Tracks participating agents
Event	event_id, scenario_id, timestamp, source, event_type, details	Stores simulated telemetry
Action	action_id, event_id, agent_id, action_type, timestamp, result	Records agent actions
Evaluation	evaluation_id, detection_time, containment, recovery, damage, score	Stores Referee measurements
Report	report_id, timeline, findings, recommendations	Stores final exercise output

Table 5.3: Logical Data Entities Used for Validation

5.6 Event Processing Sequence

A simulation event is treated as a traceable unit of activity. The event should contain enough information to determine what happened, when it happened and which component generated or consumed it. The minimum useful fields are an event identifier, scenario identifier, timestamp, source, event type and descriptive details.

- Event creation: a permitted simulated action generates an event.
- Timestamping: the event is assigned a consistent simulation timestamp.
- Telemetry routing: relevant information is forwarded to the Blue Team.
- Response recording: detection and response actions are stored.
- Evaluation: the Referee links actions and outcomes to measurable metrics.
- Reporting: the timeline and final findings are generated from recorded data.

5.7 Testing Strategy

Testing is performed at multiple levels so that individual modules can be validated before complete scenario execution. Functional correctness, module communication, workflow behaviour, interface behaviour and isolation are all relevant to the prototype.

Testing Level	Purpose	Primary Verification
Unit Testing	Validate individual modules	Scenario creation, event generation, metric calculation
Integration Testing	Validate module communication	Orchestrator ↔ agents ↔ sandbox ↔ scoring
Workflow Testing	Validate complete lifecycle	Scenario → simulation → detection → response → scoring
Isolation Testing	Validate sandbox boundary	No live-target or unauthorized external activity
UI Testing	Validate dashboard behaviour	Scenario selection, events and results
Report Testing	Validate generated output	Timeline, score, findings and recommendations

Table 5.4: Testing Levels

5.8 Functional Test Cases

The following test cases are derived directly from the requirements and end-to-end workflow. The Status column is intentionally left as a validation field because verified execution evidence was not available in the project material used to prepare this chapter.

ID	Test Case	Expected Result	Status
TC-01	Scenario selection	Scenario and difficulty load correctly	To be validated
TC-02	Sandbox initialization	Isolated state is created with synthetic data	To be validated
TC-03	Agent initialization	Orchestrator, Red, Blue and Referee initialize	To be validated
TC-04	Controlled simulation	Permitted action creates a simulated event	To be validated
TC-05	Telemetry generation	Event contains timestamp, source and details	To be validated
TC-06	Blue Team detection	Telemetry is received and detection is recorded	To be validated
TC-07	Response handling	Response and containment are recorded	To be validated
TC-08	Referee evaluation	Metrics are calculated from exercise data	To be validated
TC-09	Score generation	Composite score is generated	To be validated
TC-10	Timeline generation	Events appear chronologically	To be validated
TC-11	Report generation	Findings and recommendations are generated	To be validated
TC-12	Safety boundary	Simulation remains confined to sandbox	To be validated

Table 5.5: Functional Test Cases

5.9 Security and Isolation Testing

Isolation is a core requirement of HavenSec Arena. The project scope excludes attacks against real production systems, public targets, unauthorized infrastructure, real credentials and uncontrolled external networks. Boundary validation should therefore be performed before broader scenario testing.

Check	Validation Objective	Expected Condition
Sandbox boundary	Verify simulation processes use isolated resources	Activity remains inside configured sandbox
Synthetic data	Verify dummy data is used	No confidential or production data is required
External target protection	Verify no live-target dependency	No unauthorized external target is contacted
Agent permissions	Verify restricted interfaces	Agents cannot bypass the simulation controller
Event traceability	Verify timestamped records	Each action can be associated with an event
Failure handling	Verify component failure does not break isolation	Sandbox remains controlled

Table 5.6: Security and Isolation Validation

5.10 Performance and Resilience Metrics

The Referee component converts exercise observations into measurable indicators. These metrics are connected to the project objectives and allow repeated scenarios to be compared after the scoring rules have been fixed.

Metric	Meaning	Interpretation
Detection Time	Time between event generation and Blue Team detection	Lower generally indicates faster detection
Containment	Extent to which simulated activity is controlled	Higher indicates stronger response
Simulated Damage	Impact produced inside the scenario	Lower indicates better prevention / response
Recovery	Ability to restore the intended simulated state	Higher indicates better resilience
Composite Score	Weighted combination of normalized metrics	Repeatable summary of exercise performance

Table 5.7: Proposed Evaluation Metrics

5.11 Proposed Composite Scoring Model

For prototype evaluation, the resilience score can be represented as a weighted normalized combination of the selected indicators:

Composite Score = w■(Detection Performance) + w■(Containment) + w■(Recovery) + w■(Damage Control)

The weights should satisfy **w■ + w■ + w■ + w■ = 1.0**. Detection performance and damage control may require inverse normalization because lower detection time and lower simulated damage represent better outcomes.

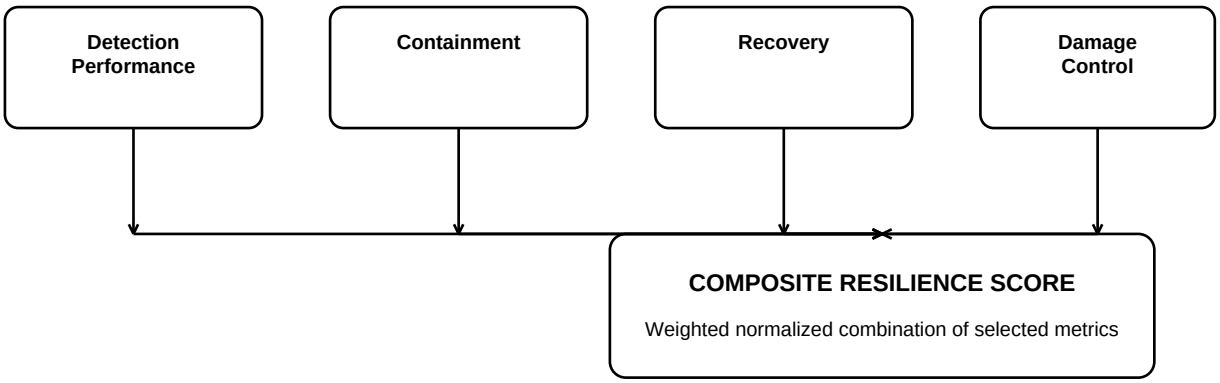


Figure 5.3: Proposed Composite Scoring Model

Important: This is an evaluation framework, not a claimed final benchmark. The exact weights, normalization method and thresholds should be frozen before official testing so that repeated runs remain comparable.

5.12 Result Recording Format

Actual results should be recorded consistently for every controlled scenario run. This format separates raw observations from the final score and makes it possible to compare repeated exercises.

Run	Scenario	Detection	Containment	Recovery	Damage	Score	Observation
1	[Name]	[value]	[value]	[value]	[value]	[value]	[finding]
2	[Name]	[value]	[value]	[value]	[value]	[value]	[finding]
3	[Name]	[value]	[value]	[value]	[value]	[value]	[finding]
4	[Name]	[value]	[value]	[value]	[value]	[value]	[finding]
5	[Name]	[value]	[value]	[value]	[value]	[value]	[finding]

Table 5.8: Actual Result Recording Format

5.13 Evidence Collection Format

Each completed test should be supported by evidence. The evidence should be tied to a specific scenario and run identifier so that screenshots and logs can be traced back to the corresponding result row.

Evidence Type	Recommended Content	Purpose
Dashboard screenshot	Scenario state, agent status and results	Confirms interface behaviour
Event log	Event ID, timestamp, source and details	Confirms telemetry generation
Response log	Detection and defensive action records	Confirms Blue Team response
Evaluation output	Metrics and composite score	Confirms Referee calculations
Timeline screenshot	Chronological event sequence	Confirms traceability
Generated report	Findings and recommendations	Confirms reporting workflow
Sandbox evidence	Container / configuration information	Supports isolation validation

Table 5.9: Recommended Evidence Collection

5.14 Expected Prototype Observations

- Scenario configuration should determine the initial simulation state.
- Red Team activity should generate controlled events without requiring a live target.
- Blue Team analysis should consume generated telemetry and record a response.
- Referee evaluation should use timestamps and recorded actions rather than subjective scoring alone.
- The incident timeline should explain the exercise chronologically.
- The final report should identify measurable weaknesses and improvement opportunities.

5.15 Problems Encountered and Corrective Approach

The following issues are practical risks for a multi-agent simulation prototype. They are presented as corrective controls rather than as claims that every issue has already occurred during execution.

Potential Issue	Impact	Corrective Approach
Agent coordination failure	Incomplete scenario execution	Use explicit interfaces and state tracking
Missing telemetry	Blue Team cannot evaluate event	Validate event schema and timestamps
Inconsistent scoring	Runs cannot be compared reliably	Freeze scoring rules before testing
Sandbox configuration error	Safety boundary may weaken	Validate isolation before execution
AI output inconsistency	Agent behaviour may vary	Use constrained prompts / rules and validate outputs
Incomplete reports	Findings may be unclear	Require timeline, metrics and recommendations

Table 5.10: Potential Issues and Corrective Measures

5.16 Implementation Limitations

The prototype remains subject to the limitations identified in the earlier chapters. Scenario realism and scenario quantity are constrained by available development time and resources. Synthetic data cannot reproduce every characteristic of enterprise telemetry, and a simplified sandbox may not represent the complexity of a production environment.

The AI component may assist with reasoning and orchestration, but it should not be treated as an autonomous authorization mechanism for real-world offensive activity. AI-generated decisions require validation and must remain subject to the simulation controller and safety boundaries.

Because the available project material contains architecture and validation plans rather than production-scale benchmark data, performance claims should only be made after actual controlled benchmark runs are recorded.

5.17 Traceability and Reproducibility

A repeatable academic prototype requires each run to have a stable scenario definition, controlled initial state, timestamped events and a fixed scoring configuration. Reproducibility is strengthened when the same scenario can be reset and executed again without changing the evaluation criteria.

- Assign a unique identifier to each scenario run.
- Record the scenario configuration before execution.
- Capture timestamps for generated events and responses.
- Preserve the evaluation parameters used for scoring.
- Store the final report together with the corresponding run evidence.

5.18 Validation Summary

Chapter 4 defined the complete conceptual workflow consisting of an isolated sandbox, Orchestrator, Red Team, Blue Team, Referee, timeline and reporting layer. This chapter converts those design elements into measurable verification points.

Area	Validation Target	Evidence to Attach
Scenario Management	Scenario can be selected and initialized	Dashboard screenshot / log
Sandbox	Simulation is isolated and uses dummy data	Container / configuration evidence
Red Team	Controlled event is generated	Event log / dashboard
Blue Team	Detection and response are recorded	Detection / response log
Referee	Metrics and score are calculated	Evaluation output
Timeline	Events are chronological	Timeline screenshot
Reporting	Findings and recommendations are generated	Generated report

Table 5.11: Validation Summary and Required Evidence

5.19 Acceptance Criteria

The prototype can be considered functionally ready for academic demonstration when the following conditions are satisfied:

- A scenario can be selected and initialized consistently.
- The sandbox can be created using synthetic or dummy data.
- Red Team, Blue Team and Referee components can participate in one controlled workflow.
- Simulation events are timestamped and traceable.
- Blue Team detection and response are recorded.
- Referee metrics are calculated from recorded evidence.
- A repeatable resilience score is produced using fixed scoring rules.
- The incident timeline and report are generated from the same exercise data.
- Isolation testing demonstrates that the exercise does not depend on live targets or uncontrolled infrastructure.

5.20 Chapter Summary

This chapter described the implementation structure, end-to-end workflow and validation framework for HavenSec Arena. The implementation is organized around scenario management, orchestration, controlled Red Team simulation, Blue Team defense, Referee scoring, incident-timeline generation and reporting.

Functional, integration, workflow, isolation, UI and reporting tests were defined to verify the proposed system. A structured result-recording method was also established using detection time, containment, simulated damage, recovery and a composite resilience score.

CHAPTER 5: IMPLEMENTATION, TESTING AND RESULTS

5.21 Final Result Interpretation Framework

Once actual runs are completed, the result analysis should focus on measurable evidence rather than subjective impressions. A single score is useful for comparison, but the underlying metrics should always be retained because two exercises can produce similar scores for very different reasons.

Observation Pattern	Possible Interpretation	Recommended Follow-up
Fast detection + strong containment	Effective defensive response	Verify repeatability across runs
Fast detection + weak containment	Detection works but response is insufficient	Review response workflow and containment controls
Slow detection + low damage	Limited impact but delayed visibility	Improve telemetry and detection logic
High recovery + high damage	Recovery works after substantial impact	Review prevention and containment
Inconsistent score across identical runs	Non-deterministic workflow or scoring	Check state reset, timestamps and scoring rules

Table 5.12: Result Interpretation Guide

5.22 Recommended Screenshots for Final Report

The final project report should include screenshots only after the corresponding functionality has been executed and verified. Recommended evidence includes:

- Scenario selection and configuration screen.
- Dashboard showing active Red Team, Blue Team and Referee states.
- Sandbox or container status demonstrating the controlled environment.
- Event / telemetry stream with timestamps.
- Blue Team detection and response output.
- Referee evaluation and resilience score.
- Incident timeline.
- Generated security report and recommendations.

5.23 Final Conclusion of Chapter

HavenSec Arena can be evaluated as a controlled academic cyber-resilience prototype by demonstrating a repeatable simulation-to-evaluation workflow while maintaining a strict sandbox boundary. The implementation framework presented here provides the structure required to convert the proposed architecture into verifiable evidence.

No unsupported numerical benchmark has been introduced in this chapter. The final project version should replace the validation placeholders with actual observations, execution logs, screenshots and measured results. Where implementation differs from the proposed architecture, those differences should be documented explicitly.

> **Submission guidance:** Before final printing, replace every “To be validated”, “[value]”, “[Name]” and “[finding]” placeholder with evidence from the actual prototype. This keeps the report technically defensible and aligned with the demonstrated system.